



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»
РТУ МИРЭА

ДИСЦИПЛИНА	Программирование управляемых технических систем (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	ПТИП (укажите название учебного института)
КАФЕДРА	Индустриального программирования (укажите полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Материалы для практических занятий (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Закожурников Сергей Сергеевич (фио)
СЕМЕСТР	2, 2026 (указать семестр обучения, учебный год)

Практическая работа

Изучение элементов

Управление состояниями элементов

Правила часто используются для управления состоянием объекта, например, для включения и выключения света при определенных условиях. Две команды могут изменить значение или состояние элемента в рамках правил:

- `MyItem.postUpdate(<new_state>)` — Изменить статус элемента, не вызывая каких-либо неявных действий. Может использоваться для отражения изменений, которые могут быть вызваны другими способами.
- `MyItem.sendCommand(<new_state>)` — Изменение статуса элемента и инициирующее возможные дальнейшие действия, например, отправку команды на связанное устройство / привязку.

По отношению к событийному правилу управление в сценариях запускает команды `postUpdate` и `sendCommand`.

Осторожно: в большинстве случаев правило с триггером `received update` срабатывает впоследствии отправки команды `sendCommand` в следующих ситуациях:

- `openHAB` автоматически обновляет статус элементов, для которых определение элемента не содержит `autoupdate=>false`;

Вещь отправляет элементу обновление статуса. Помимо конкретных команд управления методы `MyItem.sendCommand(<new_state>)` и `MyItem.postUpdate(<new_state>)` так же доступны и основные команды управления `sendCommand(MyItem, <new_state>)` и `postUpdate(MyItem, <new_state>)`. Обычно рекомендуется использовать конкретные команды.

Сравнение `MyItem.sendCommand(<new state>)` и `sendCommand(MyItem, <new state>)`

Использование методов `MyItem.sendCommand(<new_state>)` и `MyItem.postUpdate(<new_state>)` обычно предпочтительно. В этих методах объекты могут быть разных типов. А методы `sendCommand(MyItem, "<new_state>")` и `postUpdate(MyItem, "<new_state>")` напротив могут иметь в качестве аргумента только строку.

Причины этого кроются в Java — языке объектно-ориентированного программирования, на котором написан `openHAB`. Java и Rules DSL имеют два базовых типа: Примитивы и Объекты. Начинающиеся со строчной буквы названия типов после `val` или `var`, например `var int`, указывает на примитивный тип. Начинающийся с заглавной буквы тип данных, например, `var Number`, подразумевает Объект. Объекты являются более сложными сущностями по сравнению с примитивами.

У объектов есть специальные методы, которые могут автоматически

выполнять многие необходимые преобразования типов. При использовании `MyItem.sendCommand(new_state)` или `MyItem.postUpdate(new_state)` в большинстве случаев `new_state` преобразуется в тип объекта `myItem`.

Директива `sendCommand(MyItem, new_state)` не обладает такой гибкостью. Например, если `new_state` имеет примитивный тип (например, `var int new_state = 3`) а `myItem` является объектом типа диммер:

- следующая команда **недопустима**: `sendCommand(MyItem, new_state);`
- при этом команда `MyItem.sendCommand(new_state)` **будет работать**.

Использование `MyItem.postUpdate(new_state)` или `MyItem.sendCommand(new_state)` создают более стабильный код. Это, безусловно, лучший способ избежать большинства проблем. Этот синтаксис гарантирует, что любое преобразование (типизация) `new_state` выполняется наиболее подходящим для `myItem`. **Исключение:** действия полезны, когда имя элемента доступно только в виде строки. Например, если имя элемента для получения обновления или команды было вычислено в правиле путем построения строки, как в таблице 11.

Таблица 11

```
val index = 5
sendCommand("My_Lamp_" + index, ON)
```

Использование состояний элементов в правилах

Часто требуется вычислить другие значения из состояний элемента или сравнить состояния элемента с другими значениями. В `openHAB` каждый элемент имеет состояние. Состояние элемента — это сам объект, к которому можно получить доступ с помощью `MyItem.state`. Полный и актуальный список типов элементов в настоящее время разрешен в `OpenHAB`, а типы команд, которые может принимать каждый элемент, приведены в документации `openHAB` для элементов. Чтобы использовать состояние элемента в правилах, часто необходимо знать, какой тип состояния несет элемент и как преобразовать его в типы, которые можно использовать в таких операциях. И наоборот, чтобы использовать результат вычисления для изменения состояния элемента, может потребоваться его преобразование в подходящий тип.

В этом разделе проводится различие между типом команды и типом состояния. Для удобства чтения можно просто добавить «тип» в конец типа команды, получая, таким образом, тип состояния. Например, цветовой элемент может получать `OnOffType`, `IncreaseDecreaseType`, `PercentType` или `HSBType`. Следовательно, все следующие действительные команды, которые можно отправить объекту цвета, описанные в таблице 12.

Таблица 12

```
MyColorItem.sendCommand(ON)
MyColorItem.sendCommand(INCREASE)
MyColorItem.sendCommand(new PercentType(50))
```

```
MyColorItem.sendCommand(new HSBType(new DecimalType(123), new PercentType(45), new PercentType(67)))
```

Альтернативный способ указать или обновить состояние элемента — использовать специально отформатированные строки. В разделе документации элемента, посвященном форматированию, подробно описаны требования к форматированию. Несмотря на то, что многие элементы принимают команды и обновления различных типов, каждый сохраняет свое состояние внутри, используя только один тип. Color Item из приведенного выше примера будет принимать различные типы команд, но вернет только HSBType.

Группы могут быть объявлены с любым типом элемента, и внутреннее состояние группы будет соответствовать этому типу. Например, Group:Switch вернет OnOffType для своего состояния. Каждый тип состояния предоставляет ряд удобных методов, которые значительно облегчают преобразование и вычисления. Есть два способа изучить эти методы: При использовании openHAB VS Code Extension нажатие комбинации клавиш <ctrl><space> показывает список доступных методов.

Посмотрите JavaDocs для данного типа. Например, JavaDoc for HSBType показывает методы getRed, getBlue, и getGreen. Эти три метода могут быть вызваны в Rules-DSL без части своего имени, например, (MyColorItem.state as HSBType).red. Они получают состояние MyColorItem и приводят его к типу HSBType что бы можно было применять методы свойственные HSBType.

Работа с состояниями элементов: преобразования

Преобразование Item.state в String

Все состояния элемента можно преобразовать в строку, вызвав MyItem.state.toString.

Цветовой элемент

Цвет сохраняется как **HSBType**. HSB означает Оттенок (Hue), Насыщенность (Saturation) и Яркость (Brightness). Часто желаемый цвет отображается в виде значения RGB (красный, зеленый, синий). Следующий код можно использовать для отправки значения RGB в элемент цвета. Код описан в таблице 13.

Таблица 13

```
import java.awt.Color

// Создаём элемент newColor
// где red, blue и green целые числа от 0 до 255
val newColor = new Color(red, blue, green)

// Сохраняем элемент
MyColorItem.sendCommand(new HSBType(newColor))
```

Когда отдельные значения цвета из HSBType извлекаются как

PercentType, необходимо разделить PercentType на 100 и умножить на 255, чтобы получить стандартный 8-битный цветовой канал RGB. Соответственно, для 16- или 32-битного представления PercentType необходимо разделить на 100 и умножить на 65535 ($2^{16}-1$) или 4294967295 ($2^{32}-1$) соответственно. Пример преобразования для 8-битного представления показан в таблице 14.

Таблица 14

```
// В теле правила
val red = (MyColorItem.state as HSBType).red / 100 * 255
val green = (MyColorItem.state as HSBType).green / 100 * 255
val blue = (MyColorItem.state as HSBType).blue / 100 * 255
```

Элемент Contact

Элемент типа контакт имеет тип OpenCloseType. OpenCloseType — это перечисление. Он может быть преобразован из Открыто и Закрыто в 1 и 0 следующим кодом: `val contactNum = if (MyContactItem.state == OPEN) 1 else 0`

Элемент DateTime

Элемент DateTime имеет тип DateTimeType, который внутри содержит ZonedDateTime объект Java. Подробное описание изложено в коде таблицы 15.

Таблица 15

```
// Получение epoch (Unix-эпоха - число миллисекунд от 00:00:00 UTC 1 января
1970 года) из DateTimeType
val Number epoch = (MyDateTimeItem.state as
DateTimeType).zonedDateTime.toInstant.toEpochMilli

// Получение epoch bp Java ZonedDateTime
val Number nowEpoch = now.toInstant.toEpochMilli

// Преобразование DateTimeType в Java ZonedDateTime
val javaZonedDateTime = (MyDateTimeItem.state as
DateTimeType).zonedDateTime

// Преобразование Java ZonedDateTime в DateTimeType
val DateTimeType date = new DateTimeType(now)
```

В некоторых случаях необходимо преобразовать временную метку эпохи в удобочитаемую и / или сохранить ее в DateTimeType и DateTime Item. Вот возможность сделать это с помощью SimpleDateFormat, как показано в таблице 16.

Таблица 16

```
import java.time.format.DateTimeFormatter

// Преобразование Unix-эпохи в понятное людям время
val DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("yyyy-MM-dd'T'HH:mm:ss.SSSZ")
val long epoch = now.toInstant.toEpochMilli
```

```

val          ZonedDateTime          zdt          =
ZonedDateTime.ofInstant(Instant.ofEpochMilli(epoch),
ZoneOffset.UTC);
val String dateTimeString = zdt.format(formatter)

// Преобразование обратно в нормальное время DateTimeType
val DateTimeType dt = DateTimeType.valueOf(dateTimeString)

// Преобразование состояние элемента типа DateTimeType в строку
val          String          datetime_string          =
DateTime_Item.state.format("%1$td.%1$tm.%1$ty %1$tH:%1$tM")

```

ZonedDateTime предоставляет ряд полезных методов для сравнения даты и времени вместе и / или извлечения частей даты. Методы описаны в таблице 17.

Таблица 17

```

// Проверка на корректность времени с типом
DateTimeType
if(now.isBefore((MyDateTimeItem.state as
DateTimeType).zonedDateTime)) ...

// А может быть всё уже пропало типа DateTimeType
if(now.isAfter((MyDateTimeItem.state as
DateTimeType).zonedDateTime)) ...

// узнаёт который час у типа DateTimeType
val hour = (MyDateTimeItem.state as
DateTimeType).zonedDateTime.hour

```

Элемент Dimmer

Элемент **Dimmer** (регулятор) имеет тип **PercentType**. **PercentType** может быть приведен к типу **java.lang.Number** и обрабатываться как **java.lang.Number**, где **Number** представляет любой тип числового значения. Язык правил поддерживает выполнение математических и логических операций с числами. Объект **Number** поддерживает методы, описанные в таблице 18 получения примитивных версий этого числа, если это необходимо.

Таблица 18

```

// Загружаем данные из элемента в переменную
val dimVal = MyDimmerItem.state as Number
// как целое число
val int dimAsInt = dimVal.intValue
// и как число с плавающей запятой
val float dimAsFloat = dimVal.floatValue

```

Если преобразование из или в шестнадцатеричные значения необходимо, могут быть полезны следующие примеры, показанные в таблице 19.

Таблица 19

```

// преобразовать hex_code (число, выраженное в шестнадцатеричной системе) в
числовой тип
val dimVal = Integer.parseInt(hex_code, 16) as Number
// для очень больших large_hex_codes другой вариант

```

```
val dimVal = Long.valueOf(large_hex_code, 16).longValue() as Number

// и дополнительный пример преобразования integer_value в строку hex_code
var String hex = Long.toHexString(integer_value);
```

Элемент Location

Элемент Location имеет тип **PointType**. PointType содержит два или три числа DecimalType представляющих широту и долготу в градусах и опционально высоту в метрах. Примеры работы данного элемента показаны в таблице 20.

Таблица 20

```
// Создание
val location = new PointType(new DecimalType(50.12345), new
DecimalType(10.12345))
// Создание из строки. ВНИМАНИЕ: нельзя ставить пробел после запятой
val PointType home = new PointType("12.121212,123.123123")

// Загрузка из элемента
val PointType location = Device_Coordinates.state as PointType
```

Числовые элементы

Числовые элементы имеют тип **DecimalType** или **QuantityType** в случае если к кроме значения сохраняется размерность. например, Number:Temperature. В правилах размерность добавляется к числу при помощи символа |, где размерность указывается в кавычках если оно содержит символы отличные от букв и цифр, например, 10|m, but 20|"km/h". Некоторые часто используемые единицы не требуют кавычек °C, °F, Ω, °, %, m² and m³ (зачем эти исключения?), например можно написать 20|»°C», но 20|°C тоже прокатит. Типы DecimalType и QuantityType так же являются java.lang.Number и все преобразования описанные выше для Dimmer так же применимы к элементу Number. Некоторые часто используемые преобразования описаны в таблице 21.

Таблица 21

```
//Для состояний DecimalType::
// преобразует целое число в строку содержащую шестнадцатиричный код
var String hex_code = Long.toHexString(integer_number)

// преобразует шестнадцатиричный код в объект типа Number
var myNumber = Integer.parseInt(hex_code, 16) as Number
// то же для large_hex_code
var myNumber = Long.parseLong(hex, 16) as Number

// преобразует hex_code в DecimalType
var          DecimalType          parsedResult          =          new
DecimalType(Long.parseLong(hex_code, 16))
// Для состояний QuantityType:
// описание переменной типа QuantityType
var myTemperature = 20|°C

// получить размерность элемента в виде текста
var myUnits = myTemperature.getUnit.toString // gives "°C"
```

```
// преобразование количественного состояния в различные единицы:
var fahrenheit = myTemperature.toUnit("°F") // will contain quantity 68°F

// количественное значение в десятичное
var myDecimal = new DecimalType(fahrenheit.doubleValue) // myDecimal ==
68
var myCentigrade = fahrenheit.toUnit("°C").toBigDecimal // 20

// доступ к скалярным значениям как int, double, float
var myInt = fahrenheit.intValue
var mydouble = fahrenheit.doubleValue
var myfloat = fahrenheit.floatValue

// проверка не является ли состояние элемента количеством
var isQuantity = myItem.state instanceof QuantityType

// сравнение количества
// Заманчиво, но конструкция if (fahrenheit > 10) НЕ РАБОТАЕТ!!
if (fahrenheit > 10|°C) { logInfo("test", "It's warm.") }
```

Одно предупреждение про `DecimalType` (оно так же применимо к `QuantityType`). Во избежание ошибок «неоднозначного вызова метода» всегда приводите состояние типа `DecimalType` к `Number`, а не наоборот. Позаботьтесь о математике, связанной с `QuantityType`. Хотя во многих случаях вы можете свободно смешивать единицы, но есть подводные камни, указанные в таблице 22, типа «грабли».

Таблица 22

```
// сложим количественные переменные разных размерностей
var miles = 2|mi
var metres = 10|m
var distance = miles + metres // результат 2.0062 mi
// Размерность у результата та же что у первого слагаемого

var area = metres * metres // результат 100 m²
// Новые подходящие единицы используются для результата

var fahr = 68|°F
var centi = 1|°C
var sumTemps = fahr + centi // результат 101.80 °F
// Результат может показаться неожиданным...
// Однако, температура всегда считается абсолютной, а не интервалом или
приращением.
// 1°C был преобразован в 33.8°F, а не в интервал 1.8°F

// Для этого есть математический трюк
var increment = fahr + centi - 0|°C // результат 69.80 °F
// "выделение нуля" фиксирует смещение между разными размерностями
```

Элемент Player

Элемент проигрыватель (player) позволяет управлять проигрыванием (например аудиоплеера) при помощи команд Play, Pause, Next, Previous, Rewind и Fastforward. Элемент типа проигрыватель имеет три типа, указанные в таблице 23, с определёнными командами.

Таблица 23

Тип Состояния	Команды
PlayPauseType	PLAY, PAUSE
RewindFastforwardType	REWIND, FASTFORWARD
NextPreviousType	NEXT, PREVIOUS

Эти три типа могут быть преобразованы по типу контакта из Открыто и Замкнуто в 1 и 0 при помощи этого простого скопированного кода (из раздела про контакты). Получение состояния из элемента выглядит так: `val int Playing = if (MyPlayerItem.state == PLAY) 1 else 0.`

Тип Rollershutter (жалюзи)

В дополнение к командам свойственным Dimmer элемент Rollershutter понимает StopMoveType с командами STOP и MOVE.

Элемент Строка (String)

Чтобы преобразовать состояние элемента, содержащего StringType, можно вызвать метод toString. Получения строки из элемента выглядит так: `val stateAsString = MyStringItem.state.toString.` Если элемент возвращает строку, содержащую значение в виде шестнадцатеричного числа, его можно преобразовать в целое число из шестнадцатеричного числа так: `val itemvalue = new java.math.BigDecimal(Integer.parseInt(myHexValue, 16)).`

Элемент Выключатель (Switch)

Элемент Switch имеет OnOffType. OnOffType — это перечисление. Можно преобразовать из ВКЛ и ВЫКЛ в 1 и 0 с помощью кода, подобного следующему: `val SwitchNum = if (MySwitchItem.state == ON) 1 else 0.`

Взаимодействуя с состояниями Item, необходимо внимательно понимать разницу между объектами и примитивами. Как и все объектно-ориентированные компьютерные языки, Java и Rules DSL реализовали концепцию наследования. Однако наследование применяется только к объектам и не применяется к примитивам; примерами примитивов являются integer и boolean. Наследование позволяет взять существующий тип объекта, называемый классом, и добавить к нему, чтобы превратить его во что-то другое. Это «нечто иное» становится Дочерним от исходного Класа, родителем. Ребенок по-прежнему может делать все, что может делать родитель. Базовый класс верхнего уровня для всех объектов в Java и DSL правил называется просто Object.

В дополнение к другим полезным вещам в классе Object реализован метод с именем toString. А поскольку Object является родительским для

всех объектов, ВСЕ классы также реализуют `toString` метод. Однако примитивы не наследуются от *Object*. Они ни от чего не наследуются, и у них вообще нет никаких методов, включая отсутствие метода *toString*.

Объекты обычно оснащены гораздо большим количеством методов преобразования типов, в то время как примитивы не поддерживают преобразование типов. Это различие очень важно при попытке использовать результат вычисления и применить его к состоянию статьи. Это `sendCommand` универсальное действие, которое должно работать со всеми типами элементов. Действия поддерживают только два аргумента `String`, поскольку все объекты будут поддерживать преобразование `toString.sendCommand(MyItem, new_state)` будет автоматически использовать `MyItem.toString` метод для преобразования `MyItem` в `String`. Он также попытается сделать это со вторым аргументом, если `new_state` это еще не `String`. Однако, если второй аргумент является примитивом, а не объектом, он не несет метода `toString`. Таким образом, правила DSL не могут быть преобразованы `new_state` в строку. Как следствие, использование `sendCommand(MyItem, primitive)` при использовании примитива в качестве второго аргумента почти всегда не удастся.

Различный синтаксис для общего и специфического различается и приведен в таблице 24 ниже.

Таблица 24

Generic (Action)	Specific (Method)
<code>postUpdate(MyItem, new_state)</code>	<code>MyItem.postUpdate(new_state)</code>
<code>sendCommand(MyItem, new_state)</code>	<code>MyItem.sendCommand(new_state)</code>

Преимущество использования объектов над примитивами проявляется в следующих преобразованиях типов, которые автоматически вызываются объектом в зависимости от контекста. Использование метода `MyItems.sendCommand()`, которым владеет `MyItem`, будет использовать `sendCommand` метод, который подходит для выполнения необходимых преобразований типов. Например, `NumberItem` класс будет иметь `sendCommand(int)`, `sendCommand(long)`, `sendCommand(float)`, `sendCommand(double)`, `sendCommand(Number)`, `sendCommand(DecimalType)`, и `sendCommand(String)` метод. Каждый из этих отдельных методов индивидуально написан для обработки всех этих различных типов объектов. `MyItem` автоматически применит метод,

соответствующий типу аргумента.

Задание:

1. Создать цветовой элемент
2. Создать элемент Contact
3. Создать элемент DateTime
4. Создать элемент Dimmer
5. Создать элемент Location
6. Создать числовой элемент
7. Создать элемент Player
8. Создать элемент Rollershutter
9. Создать элемент Строка
10. Создать элемент Выключатель